RETAILMART V3 · SQL

Window Functions Part 2 -- Aggregation & Frames

— Siraj Sir

📱 WhatsApp Announcement

Window Functions Part 2 (Aggregation & Frames)

Yesterday you ranked rows. Today you ACCUMULATE across rows. One tiny syntax change -- adding ORDER BY inside OVER() -- turns SUM into a RUNNING TOTAL. This powers every cumulative revenue chart, moving average, and trend line you have ever seen in a dashboard.

Part 1: Window Aggregates -- SUM, AVG, COUNT with OVER
Part 2: Running Totals -- ORDER BY inside OVER changes everything
Part 3: Window Frames -- ROWS BETWEEN for moving averages

Running totals and moving averages appear in every analyst dashboard. The CFO's monthly cumulative revenue chart? Running total. The ops team's 7-day order trend? Moving average. Both are one-line window functions.

Prerequisites: Window Functions Part 1 (OVER, PARTITION BY, ranking). Today builds directly on that foundation.

RECAP

Recap

Yesterday we learned the four ranking window functions:

- OVER(PARTITION BY ...) -- aggregate without collapsing rows
- ROW_NUMBER -- unique sequence, deduplication, pagination
- RANK / DENSE_RANK -- ties with/without gaps, Nth highest
- NTILE(N) -- quartile/decile/percentile bucketing
- Top N per Category -- CTE + ranking + WHERE rn <= N

Today we use the SAME OVER() clause with aggregate functions (SUM, AVG, COUNT) -- but with a critical twist: adding ORDER BY inside OVER changes an aggregate from a group total to a RUNNING total.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: Window Aggregates	30 min	SUM/AVG/COUNT with OVER(PARTITION BY), Percentage of total per row, Comparison to group average
Part 2: Running Totals	40 min	ORDER BY inside OVER = running total, Cumulative revenue by month, Running total per partition with reset
Part 3: Window Frames (ROWS BETWEEN)	50 min	ROWS BETWEEN syntax, Moving averages (7-day, 30-day), Bounded vs unbounded frames

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Basic Queries
- ✓ Data Types
- ✓ Constraints
- ✓ Filtering
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregates
- ✓ Joins Part 2
- ✓ Self, Cross & Sets
- ✓ Subqueries Part 1
- ✓ CTEs & Correlated
- ✓ DB Concepts
- ✓ Review & Practice
- ✓ Window Functions P1
- 17 **Window Functions P2** ← HERE

STORY

The CFO's Cumulative Revenue Chart

Every month the CFO opens her dashboard and looks at one chart: cumulative revenue over time. January shows 12M. February shows 25M (12M + 13M). March shows 39M (12M + 13M + 14M). Each month stacks on top of the previous months.

The old analyst computed this with a correlated subquery: for each month, SUM all revenue from months <= current month. On 12 months of data it was fine. On 36 months it crawled. On 60 months it timed out.

The new analyst replaced it with one line: SUM(revenue) OVER (ORDER BY month). The database scans the data ONCE, accumulating as it goes. Same result. 100x faster.

The secret: adding ORDER BY inside OVER() changes an aggregate from 'total of the whole partition' to 'running total up to the current row.' One syntax change. Massive analytical power.

STORY

Every Row Gets Context Without Losing Detail

Yesterday we used OVER(PARTITION BY store_id) with AVG() to show each employee alongside their store's average salary. That was a window aggregate -- it computed an aggregate across the partition and attached the result to every row.

Today we go deeper. SUM() OVER gives you group totals on every row -- perfect for calculating each product's percentage of category revenue. COUNT() OVER gives you group counts alongside each detail row.

The rule: OVER() without ORDER BY computes the aggregate over the ENTIRE partition (group total). OVER() WITH ORDER BY computes a running aggregate up to the current row. Part 1 covers the first case. Part 2 covers the second.

DEFINITION

Window Aggregates (No ORDER BY = Group Total)

SUM() OVER(PARTITION BY ...):

Adds a column with the group total to every row. Every row in the partition gets the SAME total value. Use for: percentage of total, comparison to group total.

AVG() OVER(PARTITION BY ...):

Adds the group average to every row. Use for: comparing each row to its group average (above/below), deviation from mean.

COUNT() OVER(PARTITION BY ...):

Adds the group row count to every row. Use for: knowing group size alongside detail, calculating ratios.

Key rule:

Without ORDER BY in OVER(), the aggregate covers the ENTIRE partition. With ORDER BY, it becomes a RUNNING aggregate. This distinction is Part 2.

Window Aggregate vs GROUP BY -- Same Result, Different Shape

```
-- GROUP BY: one row per category (products -> 10 rows)
SELECT dc.category_name, SUM(p.price) AS cat_total
FROM products.products p
JOIN core.dim_brand db ON p.brand_id = db.brand_id
JOIN core.dim_category dc ON db.category_id = dc.category_id
GROUP BY dc.category_name;

-- Window: ALL rows kept, total attached (all rows kept!)
SELECT p.product_name, dc.category_name, p.price,
       SUM(p.price) OVER (PARTITION BY dc.category_name)
         AS cat_total
FROM products.products p
JOIN core.dim_brand db ON p.brand_id = db.brand_id
JOIN core.dim_category dc ON db.category_id = dc.category_id;

-- Each product row now shows its category total
-- Use it for: price / cat_total = percentage of category
```

Easy: Product's Percentage of Category Revenue

EASY

SCENARIO: The category manager wants to see each product's price as a percentage of its category's total price. This tells her which products dominate their category and which are negligible.

TASK: Return product_name, category, price, category_total, and pct_of_category. Calculate the percentage by dividing price by the category total. Round to 2 decimal places.

HINT: SUM(price) OVER(PARTITION BY category) gives the category total on every row. Then price / category_total * 100 gives the percentage. No GROUP BY needed.

Answer

✅ ANSWER

```
SELECT p.product_name, dc.category_name, p.price,
       SUM(p.price) OVER (PARTITION BY dc.category_name)
         AS category_total,
       ROUND(p.price * 100.0
         / SUM(p.price) OVER (PARTITION BY dc.category_name), 2)
         AS pct_of_category
FROM products.products p
JOIN core.dim_brand db ON p.brand_id = db.brand_id
JOIN core.dim_category dc ON db.category_id = dc.category_id
ORDER BY dc.category_name, pct_of_category DESC;
```

Medium: Orders per Store vs Company Total

MEDIUM

SCENARIO: The operations team wants a report where each store shows its order count alongside the total company-wide order count and the percentage. This is for a 'contribution to business' leaderboard.

TASK: Pre-aggregate to one row per store using a CTE. Then apply two window functions: SUM() OVER() for company total (no partition = entire table), and the percentage calculation. Return store_id, store_orders, company_total, pct_contribution.

HINT: First CTE aggregates COUNT(*) per store_id. The outer query uses SUM() OVER() on those store counts to get the company-wide total. Two layers: GROUP BY in CTE, window in outer.

Answer

✅ ANSWER

```
WITH store_counts AS (
    SELECT store_id,
           COUNT(*) AS store_orders
    FROM sales.orders
    GROUP BY store_id
)
SELECT sc.store_id, sc.store_orders,
       SUM(sc.store_orders) OVER () AS company_total,
       ROUND(sc.store_orders * 100.0
         / SUM(sc.store_orders) OVER (), 2)
         AS pct_contribution
FROM store_counts sc
ORDER BY pct_contribution DESC;
```

KEY TAKEAWAYS

Window aggregates WITHOUT ORDER BY compute group totals that appear on every row. SUM() OVER(PARTITION BY X) = group total per X. SUM() OVER() = grand total (no partition). Use for percentage-of-total calculations, comparing each row to its group, and adding context without losing row-level detail. No GROUP BY needed.

STORY

The One-Line Change That Creates Running Totals

Watch this carefully. Two queries. One difference.

Query A: SUM(revenue) OVER (PARTITION BY region). Every row in the region shows the SAME total. This is a group total.

Query B: SUM(revenue) OVER (PARTITION BY region ORDER BY month). Each row shows a DIFFERENT total -- the sum of revenue from the FIRST month up to the CURRENT row's month. This is a running total.

The only difference? Adding ORDER BY. When you add ORDER BY to OVER(), PostgreSQL implicitly applies a window frame: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. Translation: 'sum everything from the start of the partition up to this row.'

DEFINITION

Running Total = ORDER BY Inside OVER()

Without ORDER BY:

SUM(x) OVER (PARTITION BY grp) = the SAME group total on every row. Static. Same number repeated.

With ORDER BY:

SUM(x) OVER (PARTITION BY grp ORDER BY col) = running total that grows row by row. Each row shows the cumulative sum from the partition start up to that row's position.

Why it works:

ORDER BY triggers a default window frame: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. This means 'include all rows from the beginning of the partition up to and including the current row.'

Group Total vs Running Total -- Visual

```
month | revenue | GROUP TOTAL      | RUNNING TOTAL
Jan   |  12M    | 50M (same!)      | 12M
Feb   |  13M    | 50M (same!)      | 25M  (12+13)
Mar   |  14M    | 50M (same!)      | 39M  (12+13+14)
Apr   |  11M    | 50M (same!)      | 50M  (12+13+14+11)

-- Group total (no ORDER BY):
SUM(revenue) OVER ()   --> same 50M on every row

-- Running total (with ORDER BY):
SUM(revenue) OVER (ORDER BY month)  --> grows each row
```

Easy: Cumulative Revenue by Month

EASY

SCENARIO: The CFO wants the classic cumulative revenue chart: month, monthly revenue, and cumulative revenue. Each month's cumulative is the sum of ALL months up to and including that month.

TASK: First aggregate monthly revenue from sales.orders in a CTE. Then use SUM() OVER(ORDER BY month) to compute the running total. Return month, monthly_revenue, cumulative_revenue.

HINT: DATE_TRUNC('month', order_date) gives the month. ORDER BY month in the OVER clause makes SUM accumulate chronologically.

Answer

✅ ANSWER

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::DATE
             AS month,
           SUM(net_total) AS monthly_revenue
    FROM sales.orders
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month,
       monthly_revenue,
       SUM(monthly_revenue) OVER (
           ORDER BY month
       ) AS cumulative_revenue
FROM monthly
ORDER BY month;
```

Medium: Running Total of Expenses per Department

MEDIUM

SCENARIO: The finance team tracks department-level expenses month by month. They want a running total that resets for each department -- HR's cumulative should not include Marketing's expenses.

TASK: From finance.expenses, compute monthly expenses per expense_category. Then apply SUM() OVER(PARTITION BY expense_category ORDER BY month) to get a running total that resets per department.

HINT: PARTITION BY expense_category makes each department its own window. The running total starts fresh for each department. ORDER BY month controls the accumulation direction.

Answer

✅ ANSWER

```
WITH dept_monthly AS (
    SELECT dec.category_name AS department,
           DATE_TRUNC('month', e.expense_date)::DATE
             AS month,
           SUM(e.amount) AS monthly_expense
    FROM finance.expenses e
    JOIN core.dim_expense_category dec
      ON e.exp_cat_id = dec.exp_cat_id
    GROUP BY dec.category_name,
             DATE_TRUNC('month', e.expense_date)
)
SELECT department, month,
       monthly_expense,
       SUM(monthly_expense) OVER (
           PARTITION BY department
           ORDER BY month
       ) AS cumulative_expense
FROM dept_monthly
ORDER BY department, month;
```

Hard: Reset Running Total at Year Boundary

HARD

SCENARIO: Finance wants cumulative revenue that resets every year. January 2025 starts at 0, accumulates through December 2025. January 2026 resets to 0 and starts accumulating again.

TASK: Extract both year and month from order dates. PARTITION BY year so the running total resets each year. ORDER BY month so it accumulates within the year. Return year, month_name, monthly_revenue, ytd_revenue.

HINT: PARTITION BY EXTRACT(YEAR FROM month) creates one window per year. The running total starts at January's revenue and grows through December.

Answer

✅ ANSWER

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::DATE
             AS month,
           SUM(net_total) AS monthly_revenue
    FROM sales.orders
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT EXTRACT(YEAR FROM month)::INT AS year,
       TO_CHAR(month, 'Mon') AS month_name,
       monthly_revenue,
       SUM(monthly_revenue) OVER (
           PARTITION BY EXTRACT(YEAR FROM month)
           ORDER BY month
       ) AS ytd_revenue
FROM monthly
ORDER BY month;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

Adding ORDER BY inside OVER() transforms an aggregate from static group total to running cumulative. SUM() OVER(ORDER BY x) = running sum. PARTITION BY + ORDER BY = running sum that resets per group. This replaces correlated subqueries for cumulative calculations.

COMMON MISTAKE

Common Mistakes with Running Totals

Mistake 1: Forgetting ORDER BY makes it a static total

SUM(revenue) OVER(PARTITION BY region) gives the SAME total on every row. Not a running total.

Fix:

Add ORDER BY: SUM(revenue) OVER(PARTITION BY region ORDER BY month).

Mistake 2: Running total does not reset where expected

SUM(revenue) OVER(ORDER BY month) gives a global running total across ALL regions and years.

Fix:

Add PARTITION BY: SUM(revenue) OVER(PARTITION BY region ORDER BY month).

STORY

The 7-Day Moving Average

The ops team's daily order dashboard has wild spikes -- weekends are low, Mondays are high, sale days are 3x normal. The raw daily numbers are noisy and hard to read.

A 7-day moving average smooths the noise. For each day, it averages that day plus the previous 6 days. The result is a smooth trend line that reveals whether orders are growing or declining.

The window frame ROWS BETWEEN 6 PRECEDING AND CURRENT ROW defines exactly this: 'look at the current row plus 6 rows before it.' Apply AVG() to this 7-row window, and you have a moving average.

DEFINITION

Window Frame Syntax -- ROWS BETWEEN

Syntax:

function() OVER (ORDER BY col ROWS BETWEEN start AND end)

Frame boundaries:

UNBOUNDED PRECEDING = first row of partition. N PRECEDING = N rows before current. CURRENT ROW = this row. N FOLLOWING = N rows after current. UNBOUNDED FOLLOWING = last row of partition.

Common patterns:

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW = running total (default). ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = 7-day moving average. ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING = 3-row centered average.

ROWS vs RANGE:

ROWS counts physical rows. RANGE considers value equality. For moving averages, always use ROWS -- it gives predictable, position-based windows.

Window Frame -- Visual

```
Row | Value | ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  1 |    10 | AVG(10)           = 10.0  (only 1 row)
  2 |    20 | AVG(10,20)        = 15.0  (only 2 rows)
  3 |    30 | AVG(10,20,30)     = 20.0  (full 3-row window)
  4 |    40 | AVG(20,30,40)     = 30.0  (slides forward!)
  5 |    50 | AVG(30,40,50)     = 40.0

-- The 'window' of 3 rows slides down the partition
-- First rows have fewer rows (partial window)
```

Frame Syntax -- All Variations (1/2)

```
-- 7-day moving average
AVG(daily_orders) OVER (
    ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
)

-- Running total (explicit default)
SUM(revenue) OVER (
    ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)

-- Full partition total (explicit)
SUM(revenue) OVER (
    ROWS BETWEEN UNBOUNDED PRECEDING
             AND UNBOUNDED FOLLOWING
)

-- 3-row centered average
AVG(value) OVER (
```

Frame Syntax -- All Variations (2/2)

```
    ORDER BY date
    ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
)
```

Easy: 7-Day Moving Average of Daily Orders

EASY

SCENARIO: The ops dashboard shows daily order counts, but the numbers are noisy. A 7-day moving average smooths the trend. For each day, average that day plus the 6 preceding days.

TASK: Aggregate daily order count from sales.orders. Apply AVG() with ROWS BETWEEN 6 PRECEDING AND CURRENT ROW. Return order_date, daily_orders, moving_avg_7d.

HINT: ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = window of 7 rows (6 before + current). Early days with fewer than 7 rows will have a smaller window automatically.

Answer

✅ ANSWER

```
WITH daily AS (
    SELECT order_date::DATE AS order_date,
           COUNT(*) AS daily_orders
    FROM sales.orders
    GROUP BY order_date::DATE
)
SELECT order_date,
       daily_orders,
       ROUND(AVG(daily_orders) OVER (
           ORDER BY order_date
           ROWS BETWEEN 6 PRECEDING
                    AND CURRENT ROW
       ), 1) AS moving_avg_7d
FROM daily
ORDER BY order_date;
```

Medium: Rolling 30-Day Customer Count

MEDIUM

SCENARIO: The marketing team wants to know how many unique customers ordered in any rolling 30-day window. This shows customer engagement trends.

TASK: For each day, count distinct customers who ordered on that day. Then compute a rolling 30-day SUM. Note: this approximates since a customer may appear in multiple days within the window.

HINT: ROWS BETWEEN 29 PRECEDING AND CURRENT ROW gives a 30-day window (29 before + current = 30 rows).

Answer

✅ ANSWER

```
WITH daily_customers AS (
    SELECT order_date::DATE AS order_date,
           COUNT(DISTINCT cust_id)
             AS unique_customers
    FROM sales.orders
    GROUP BY order_date::DATE
)
SELECT order_date,
       unique_customers,
       SUM(unique_customers) OVER (
           ORDER BY order_date
           ROWS BETWEEN 29 PRECEDING
                    AND CURRENT ROW
       ) AS rolling_30d_customer_days
FROM daily_customers
ORDER BY order_date;
```

Crazy: CFO Dashboard -- 4 Window Functions in One Query

CRAZY

SCENARIO: The CFO wants one monthly report with: monthly revenue, YTD cumulative (resets each year), 3-month moving average, and percentage of the year's total. All in a single query.

TASK: Build a CTE with monthly revenue. In the outer query, apply four window functions: (1) SUM with PARTITION BY year + ORDER BY month for YTD, (2) AVG with ROWS BETWEEN 2 PRECEDING AND CURRENT ROW for 3-month moving avg, (3) SUM with PARTITION BY year (no ORDER BY) for year total, (4) percentage = monthly / year total.

HINT: Four different OVER() clauses in one SELECT. PostgreSQL handles them all in a single pass.

Answer (1/2)

✅ ANSWER

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::DATE
             AS month,
           SUM(net_total) AS revenue
    FROM sales.orders
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month, revenue,
       SUM(revenue) OVER (
           PARTITION BY EXTRACT(YEAR FROM month)
           ORDER BY month
       ) AS ytd_revenue,
       ROUND(AVG(revenue) OVER (
           ORDER BY month
           ROWS BETWEEN 2 PRECEDING
                    AND CURRENT ROW
       ), 2) AS moving_avg_3m,
       ROUND(revenue * 100.0
         / SUM(revenue) OVER (
             PARTITION BY EXTRACT(YEAR FROM month)
         ), 2) AS pct_of_year
FROM monthly
```

Answer (2/2)

✅ ANSWER

```
ORDER BY month;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

ROWS BETWEEN defines which rows the aggregate covers. 6 PRECEDING AND CURRENT ROW = 7-row moving window. UNBOUNDED PRECEDING AND CURRENT ROW = running total. UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING = full partition. Always use ROWS (not RANGE) for predictable row-count windows.

COMMON MISTAKE

Common Mistakes with Window Frames

Mistake 1: Using RANGE instead of ROWS

RANGE BETWEEN 6 PRECEDING does NOT mean '6 rows back' -- it means '6 units of the ORDER BY value.' With dates, gaps in data give unexpected results.

Fix:

Use ROWS for a guaranteed N-row window.

Mistake 2: Forgetting the implicit frame with ORDER BY

SUM(x) OVER(ORDER BY col) uses an implicit frame: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. This is a running total, NOT a group total.

Fix:

For a static group total, omit ORDER BY: SUM(x) OVER(). Be intentional.

INTERVIEW

Running Total vs Group Total

Q: How do you create a running total in SQL?

A: SUM(column) OVER (ORDER BY sort_column). Adding ORDER BY inside OVER() makes the aggregate cumulative. Without ORDER BY, every row shows the same group total. With PARTITION BY, the running total resets per group.

INTERVIEW

Moving Average

Q: How would you compute a 7-day moving average?

A: AVG(metric) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW). ROWS BETWEEN defines a sliding window of 7 rows. Always use ROWS, not RANGE, for a guaranteed row-count window.

INTERVIEW

ROWS vs RANGE

Q: What is the difference between ROWS and RANGE in window frames?

A: ROWS counts physical row positions (3 PRECEDING = exactly 3 rows back). RANGE compares values (3 PRECEDING = values within 3 of the current). For moving averages, always use ROWS. RANGE is the default, which is why being explicit matters.

REAL WORLD

What You Achieved Today

1. Window Aggregates

SUM/AVG/COUNT OVER(PARTITION BY ...) without ORDER BY = static group total on every row. Use for percentage-of-total and comparing each row to its group.

2. Running Totals

Adding ORDER BY inside OVER() turns an aggregate into a running cumulative. PARTITION BY resets per group. Replaces correlated subqueries.

3. Window Frames (ROWS BETWEEN)

Explicit frame control for moving averages and bounded windows. ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = 7-day moving average. Always use ROWS, not RANGE.

Where This Is Used:

- CFO dashboards: Cumulative revenue, YTD tracking
- Ops teams: 7-day and 30-day moving averages
- Marketing: Rolling active customer counts
- Finance: Running expense totals per department

Window Functions Part 2 -- Aggregation & Frames

-- Group Total (No ORDER BY)

```
SUM(price) OVER(PARTITION BY category)
  -- SAME total on every row
  -- Use for: price / SUM OVER = pct
```

-- Running Total (With ORDER BY)

```
SUM(revenue) OVER(ORDER BY month)
  -- GROWS each row (cumulative)
  -- Default frame: UNBOUNDED PRECEDING
  --   to CURRENT ROW
```

-- Running Total with Reset

```
SUM(revenue) OVER(
  PARTITION BY year ORDER BY month)
  -- Resets at each year boundary
```

Window Functions Part 2 -- Aggregation & Frames

-- 7-Day Moving Average

```
AVG(metric) OVER(
  ORDER BY date
  ROWS BETWEEN 6 PRECEDING
           AND CURRENT ROW)
```

-- Frame Boundaries

```
UNBOUNDED PRECEDING  -- first row
N PRECEDING          -- N rows back
CURRENT ROW          -- this row
N FOLLOWING          -- N rows ahead
UNBOUNDED FOLLOWING  -- last row
```

-- ROWS vs RANGE

```
ROWS: physical position (predictable)
RANGE: value-based (ties grouped)
-- Always use ROWS for moving avg
```

CHALLENGE

CHALLENGE (1/4)

12 Practice Problems

Window Aggregates (1-3)

CHALLENGE

CHALLENGE (2/4)

1.: For each employee, show salary, store average, company average, and difference from store average -- all without GROUP BY.

2.: For each order, show net_total and what percentage it represents of its store's total revenue.

3.: For each store, show store_name, employee_count (via window), and total company employee count.

Running Totals (4-7)

CHALLENGE

CHALLENGE (3/4)

4.: Cumulative revenue chart: month, monthly_revenue, cumulative_revenue.

5.: Running total of expenses per expense_category that resets each year.

6.: Running count of orders per customer -- showing each order with its sequence number.

7.: Running MAX of net_total per store -- the 'record order' as of each date.

Window Frames (8-12)

CHALLENGE

CHALLENGE (4/4)

8.: 7-day moving average of daily order count.

9.: 30-day rolling SUM of revenue (each day shows past 30 days' total).

10.: 3-month centered moving average (1 PRECEDING + current + 1 FOLLOWING).

11.: Daily orders with 7-day AND 30-day moving averages side by side.

12.: (Crazy) Monthly CFO dashboard: revenue, cumulative, 3-month moving avg, pct_of_year -- 4 window functions in one SELECT.

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Basic Queries
- ✓ Data Types
- ✓ Constraints
- ✓ Filtering
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregates
- ✓ Joins Part 2
- ✓ Self, Cross & Sets
- ✓ Subqueries Part 1
- ✓ CTEs & Correlated
- ✓ DB Concepts
- ✓ Review & Practice
- ✓ Window Functions P1
- 17 **Window Functions P2** ← HERE